MoCafe v2.00 — MPI Scaling Benchmark

Abstract

Strong- and weak-scaling measurements of MoCafe's MPI parallelization on a dual-socket Xeon node, comparing the two photon-distribution modes (`par%use_master_slave` on/off). Equal-share mode scales nearly ideally up to the physical core count (94% parallel efficiency at 16 ranks, 84% at 32); beyond the 36 physical cores hyper-threading adds little. Master–slave mode pays exactly one rank—the dispatcher—and nothing more: its runtime follows the $(N_\mathrm{p}-1)/N_\mathrm{p}$ worker limit to within a few percent, so the two modes converge above $\sim$16 ranks, and under hyper-threaded contention the dynamic balancing of master–slave even wins slightly. Recommendations: use all physical cores but not the hyper-threads for production runs; at small N_p prefer equal-share unless the photon cost is strongly nonuniform.

Contents

1. Setup

1.1 Hardware and software

- Node: dual-socket Intel Xeon Gold 6254 @ 3.10 GHz, $2 \times 18 = 36$ physical cores, 72 hardware threads (hyper-threading).
- Compiler / MPI: Intel oneAPI `mpiifort` (`-ipo -O3 -no-prec-div -fp-model fast=2`), Intel MPI; one MPI rank per process, no OpenMP.
- The node was otherwise idle (load $\lesssim 1.5/72$) during the runs.

1.2 Benchmark case

A representative panchromatic transport run whose cost is dominated by the photon loop (input `scale_base.in` under `examples/scaling/`):

- SED mode (`par%use_sed`) with 40 log-spaced wavelength bins (0.0912–2000 μm), astrodust opacities;
- point source ($T_\star = 10^4$ K) in a $\tau_V = 10$ dust sphere on a 33^3 Cartesian grid;
- the J_λ path-length tally (`par%save_jlam`) active, so the measurement includes the per-rank tally and its final `MPI_ALLREDUCE` ($40 \times 33^3 \approx 1.4$ M doubles $\approx$11 MB — negligible here);
- one small observer image (33^2); no dust-emission stage.

Timings are wall-clock times of the whole `mpirun` invocation, which include a fixed $\approx$0.9 s of startup (MPI init, opacity tables, grid). Every run uses the same seed; the RNG streams differ per rank, so the physics output is statistically, not bitwise, identical between the two modes.

1.3 Protocol

- **Strong scaling**: fixed 6.4×10^7 photons; $N_\mathrm{p} = 1, 2, 4, \ldots, 64$ for equal-share and $N_\mathrm{p} = 2, 4, \ldots, 64$ for master–slave (a single rank cannot run master–slave: rank 0 only dispatches).
- **Weak scaling**: 10^6 photons per rank ($10^6 N_\mathrm{p}$ in total), same N_p series.
- Driver: `examples/scaling/run_scaling.sh`; analysis: `plot_scaling.py`; raw data: `results.csv`.

1.4 The two distribution modes

Equal share (`par%use_master_slave = .false.`) Rank r processes the photon indices $r{+}1, r{+}1{+}N_\mathrm{p}, r{+}1{+}2N_\mathrm{p}, \ldots$ All N_p ranks compute; there is no communication during the loop. Load balance relies on photons having statistically similar cost.

Master–slave (`par%use_master_slave = .true.`, default) Rank 0 hands out batches of `par%num_send_at_once` (= 10^4) photons on demand and does not itself transport photons; the $N_\mathrm{p}-1$ workers request a new batch when they finish one. This buys dynamic load balancing at the cost of one rank, so its ideal runtime is $T_1/(N_\mathrm{p}-1)$ rather than T_1/N_p.

2. Strong scaling

Table 1 lists the measured wall times; Fig. 1 shows the speed-up and parallel efficiency, both relative to the single-rank equal-share time $T_1 = 829.7$ s.

Table 1: Strong scaling: 6.4×10^7 photons, wall time T [s], speed-up $S = T_1/T$, and parallel efficiency $E = S/N_\mathrm{p}$.

	equal share			master–slave		
N_p	T [s]	S	E [%]	T [s]	S	E [%]
1	829.7	1.00	100.0	–	–	–
2	418.6	1.98	99.1	846.5	0.98	49.0
4	215.9	3.84	96.1	283.2	2.93	73.3
8	109.2	7.60	95.0	122.1	6.79	84.9
16	55.3	14.99	93.7	58.1	14.28	89.3
32	31.0	26.78	83.7	31.8	26.12	81.6
64	29.4	28.20	44.1	27.6	30.01	46.9

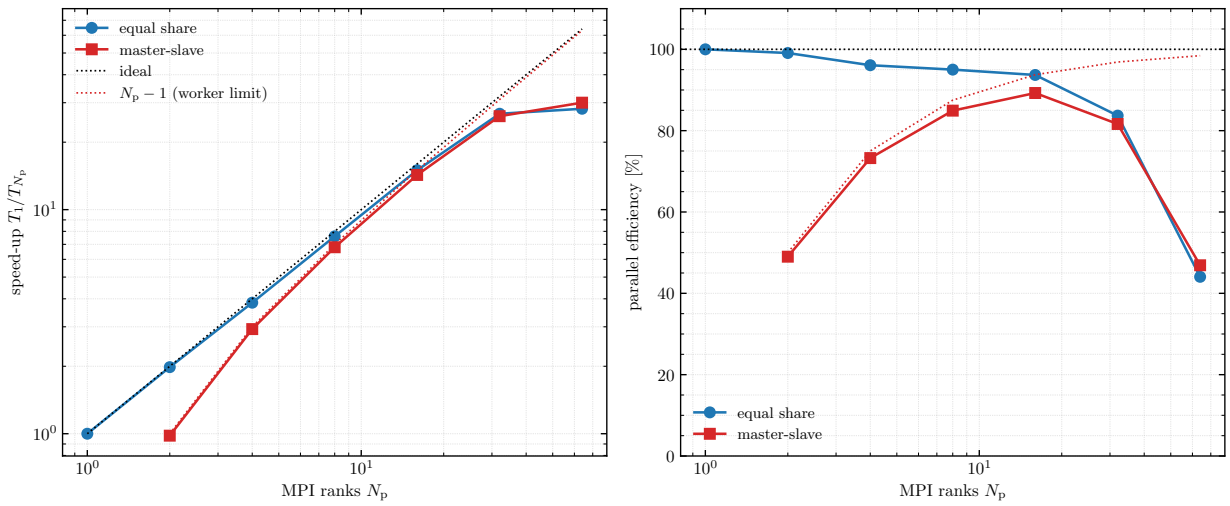

Figure 1: Strong scaling. *Left*: speed-up versus rank count with the ideal line and the master–slave worker limit $S = N_\mathrm{p} - 1$. *Right*: parallel efficiency; the red dotted curve is the worker-limit prediction $100\,(N_\mathrm{p}-1)/N_\mathrm{p}$. Both modes drop sharply at $N_\mathrm{p} = 64$, which exceeds the 36 physical cores of the node (hyper-threading).

2.1 Equal share

The photon loop is communication-free, and the efficiency stays at 94–99% up to $N_p = 16$ and 84% at $N_p = 32$. The mild decline toward 32 ranks is a hardware effect (shared caches and memory bandwidth on the two sockets), not MPI overhead: the weak-scaling runs (§3.) show the same decline at constant communication volume.

2.2 Master–slave: the cost is exactly one rank

The master–slave times follow the worker limit $T_1/(N_p-1)$ closely. Writing $T_{ms}(N_p) = (1+\epsilon)\,T_1/(N_p-1)$, the measured messaging overhead ϵ is

$$\epsilon = 2.0\%,\ 2.4\%,\ 3.0\%,\ 5.0\% \quad \text{at } N_p = 2, 4, 8, 16,$$

i.e. the dispatch messages (one small `MPI_SEND/RECV` pair per 10^4-photon batch) are essentially free. Equivalently, the ratio T_{ms}/T_{eq} tracks the ideal $N_p/(N_p-1)$: 2.02 vs. 2 at N_p=2, 1.31 vs. 1.33 at N_p=4, 1.12 vs. 1.14 at N_p=8, 1.05 vs. 1.07 at N_p=16, and 1.03 vs. 1.03 at N_p=32. The two modes therefore converge above $\sim$16 ranks, where losing one rank no longer matters.

2.3 Hyper-threading ceiling

From $N_p = 32$ to 64 the runtime barely moves (31.0 s $\rightarrow$ 29.4 s equal-share): 64 ranks oversubscribe the 36 physical cores, and two hyper-threads on one core share its execution resources. The efficiency collapse at $N_p = 64$ (44–47%) is thus a hardware ceiling, not a property of the parallelization. Interestingly, master–slave is *faster* than equal-share there (27.6 vs. 29.4 s): under hyper-threaded contention the per-photon cost becomes nonuniform across ranks, and the dynamic balancing compensates while the static equal-share split waits for its slowest rank.

3. Weak scaling

With 10^6 photons per rank the wall time would stay constant under perfect scaling. Measured (equal share): 13.7, 14.3, 14.4, 14.5, 14.7, 16.2, 29.3 s for $N_p = 1 \ldots 64$ — an efficiency of 95% at N_p = 2–16, 84% at 32, and 47% at 64 (the same hyper-threading ceiling). Master–slave starts at 50% ($N_p = 2$, one worker doing all the work) and converges to equal-share by $N_p \approx 16$, as in the strong-scaling series (Fig. 2).

The flatness of the equal-share curve up to $N_p = 32$ also confirms that the J_λ `MPI_ALLREDUCE` and the output write (both constant in this series) are negligible at this problem size. For much larger tallies ($n_\lambda \times n_{cell}$ approaching memory limits) the reduction is chunked (`jtally_mod`), and its cost should be re-measured together with the planned tally-memory work.

4. Recommendations

- **Do not exceed the physical core count.** On this node N_p = 32–36 is the sweet spot; hyper-threads add $\lesssim$5% throughput while doubling the rank count (and the tally memory, which is allocated per rank).

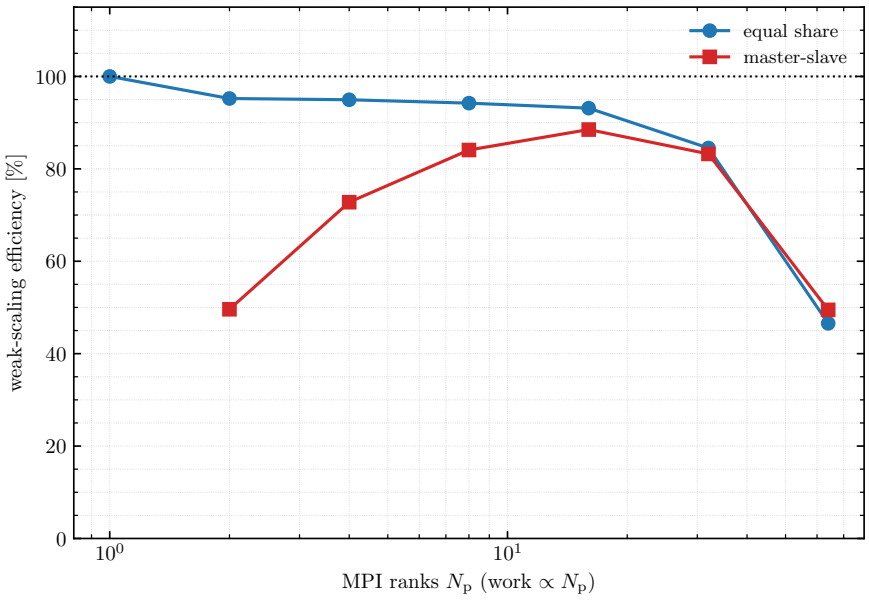

Figure 2: Weak scaling (10^6 photons per rank). Efficiency is $T_1(\mathrm{eq})/T_{N_p}$.

- $N_p \gtrsim 16$**: the mode choice hardly matters.** The master–slave penalty is one rank out of N_p and its messaging is free; above $\sim$16 ranks the two modes agree to a few percent.
- **Small N_p (2–8): prefer equal-share** for uniform work such as this benchmark — master–slave wastes 12–50% of the machine there. Use master–slave when the photon cost is strongly nonuniform (deep clouds, external illumination, mixed AMR levels), which is why it remains the default.
- **Oversubscribed or heterogeneous nodes favor master–slave**: its dynamic balancing absorbed the hyper-threading nonuniformity at $N_p = 64$, beating the static split.

5. Reproduction

- `examples/scaling/scale_base.in` — the benchmark input.
- `examples/scaling/run_scaling.sh` — runs both series and both modes, writes `results.csv`.
- `examples/scaling/plot_scaling.py` — produces `scaling_strong.pdf`, `scaling_weak.pdf`, and the rows of Table 1.